



Sistemas Distribuidos

Profesor: Oscar Manuel Ruiz Hurtado

Evidencia S08

Que la transferencia no deje dinero en el aire

Sierra García Mariana

Grupo: 02

Facultad de Ingeniería

Bitácora de Laboratorio: Microservicios y Tolerancia a Fallos

1. El problema: una transferencia incompleta

En una transferencia intervienen dos operaciones principales, primero se descuenta el dinero de la cuenta de origen y después se abona a la cuenta de destino como estas operaciones pueden ejecutarse en servicios diferentes no necesariamente forman parte de una sola transacción de base de datos.

Si el descuento se completa pero el abono falla el dinero puede quedar temporalmente descontado sin llegar a su destino, por esta razón es necesario contar con un mecanismo que permita compensar el movimiento cuando la transferencia no puede completarse.

1.1. Saldos antes del primer intento

Las primeras dos capturas muestran los saldos antes de realizar la transferencia Ana tiene $1000.00 y Beto tiene $500.00.

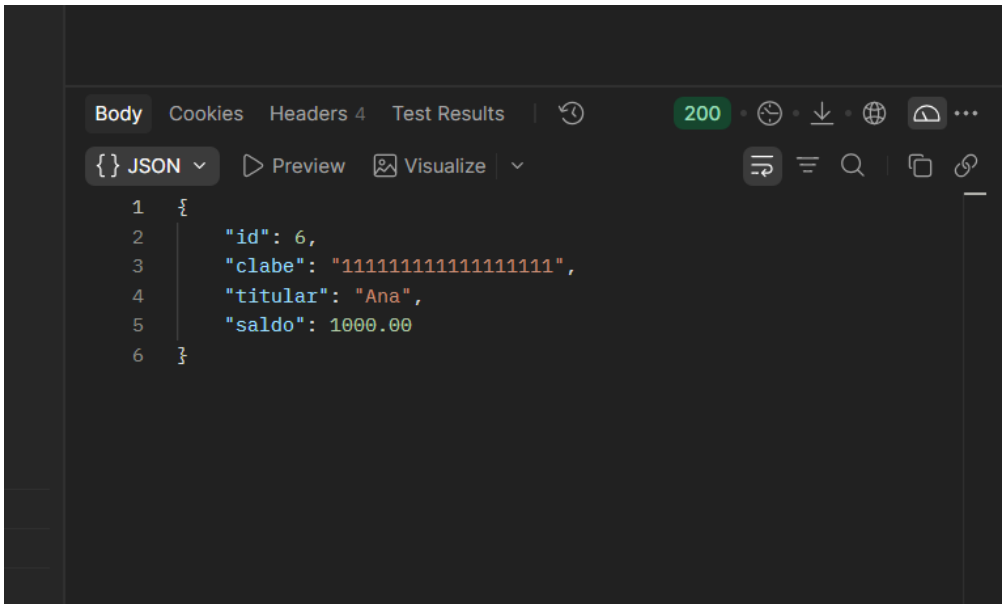


Figura 1: Consulta inicial del saldo de Ana: $1000.00.

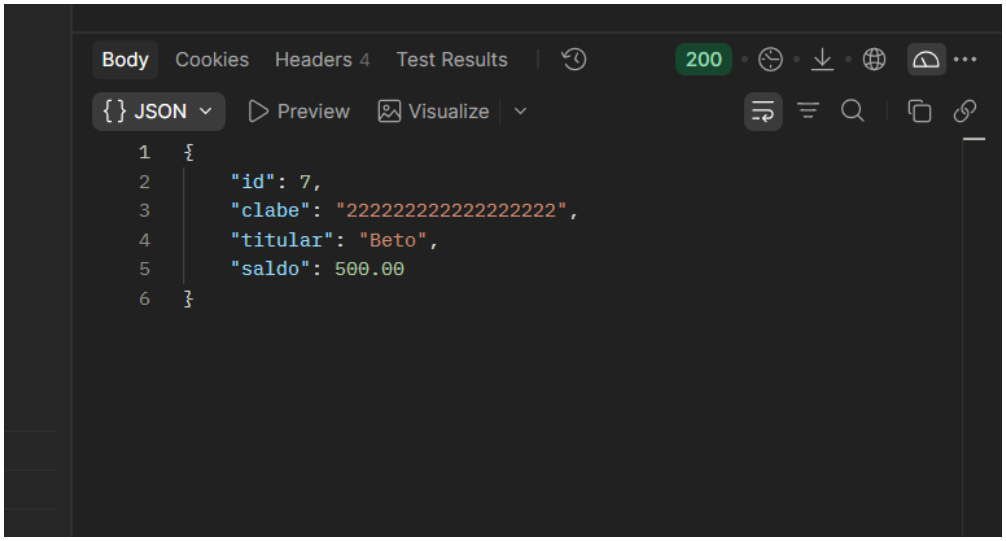


Figura 2: Consulta inicial del saldo de Beto: $500.00.

Estos valores representan el estado inicial observado en las consultas a los servicios de cuentas.

1.2. Primer intento y error 504

Durante el primer intento la API respondió con el código HTTP `504 Gateway Timeout`.

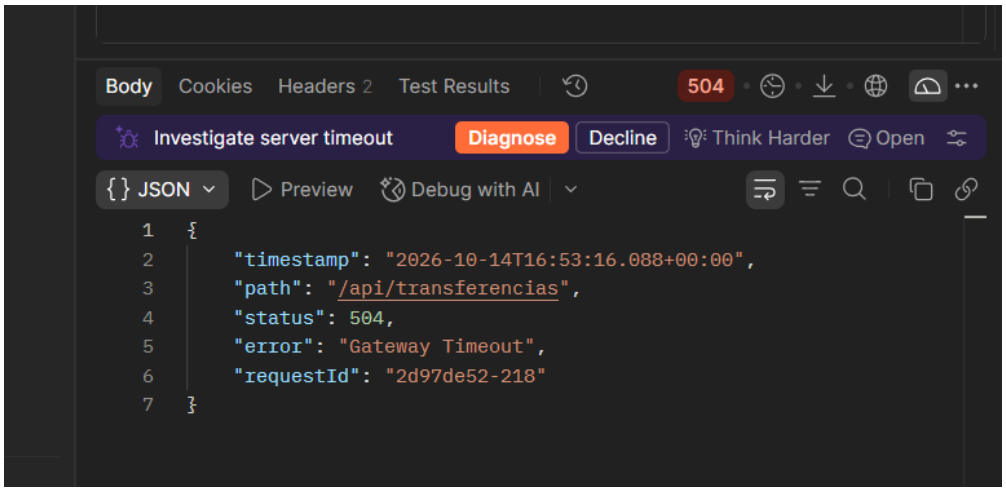


Figura 3: Primer intento de transferencia con respuesta HTTP 504.

Este error indica que la solicitud no recibió una respuesta a tiempo sin embargo no demuestra que la operación se haya cancelado el servidor pudo continuar procesando la transferencia aunque el cliente ya no estuviera esperando la respuesta.

Las siguientes capturas muestran que el saldo de Ana cambió a $800.00 y el de Beto a $700.00.

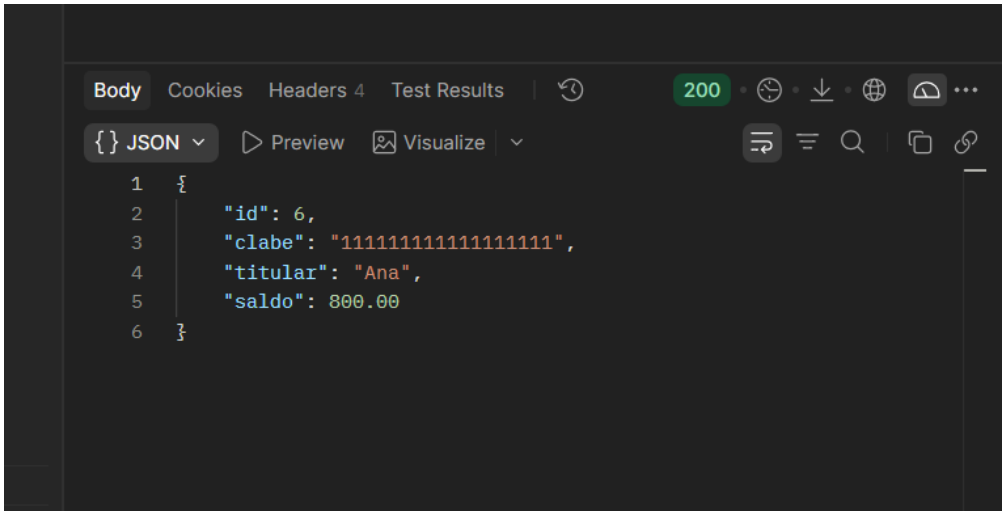


Figura 4: Saldo de Ana después del primer intento: $800.00.

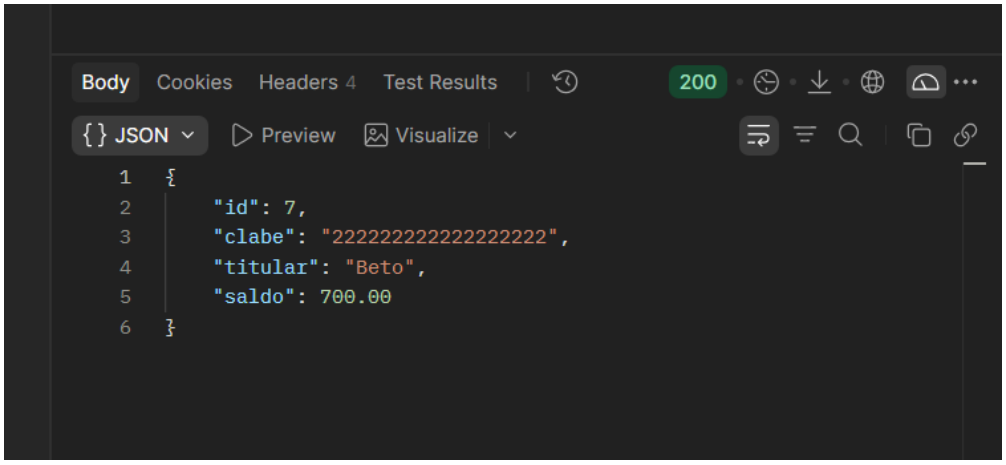


Figura 5: Saldo de Beto después del primer intento: $700.00.

Los cambios observados son consistentes con una transferencia de $200.00 que sí modificó ambas cuentas pero el problema fue que la API devolvió un error de tiempo de espera aunque los saldos ya habían cambiado.

Esto nos demuestra que un error HTTP no siempre significa que una operación de negocio no se haya ejecutado si el cliente repite la solicitud sin verificar el estado anterior podría provocar una segunda transferencia.

2. La saga de transferencia

2.1. Funcionamiento de la saga

Una saga coordina una operación distribuida mediante varias transacciones locales y si uno de los pasos falla se pueden ejecutar operaciones de compensación para contrarrestar los cambios realizados anteriormente.

En esta práctica la transferencia tiene dos pasos principales:

1. Descontar la cantidad solicitada de la cuenta de origen.
2. Abonar la misma cantidad a la cuenta de destino.

Si el abono falla después del descuento la saga debe intentar devolver el dinero a la cuenta de origen.

2.2. Pasos y compensaciones

Paso	Operación principal	Compensación
Descuento	Restar el importe de la cuenta de origen.	Devolver el importe descontado si la operación debe revertirse.
Abono	Sumar el importe a la cuenta de destino.	Si el abono falla se intenta devolver el dinero a la cuenta de origen.
Confirmación	Registrar que la transferencia terminó correctamente.	Si la compensación falla se registra como pendiente de recuperación.

Tabla 1: Pasos de la saga y sus compensaciones.

La compensación no elimina lo que ocurrió anteriormente es una nueva operación que busca recuperar el estado de las cuentas

2.3. Diagrama de estados de la transferencia

El siguiente diagrama representa el flujo esperado de una transferencia

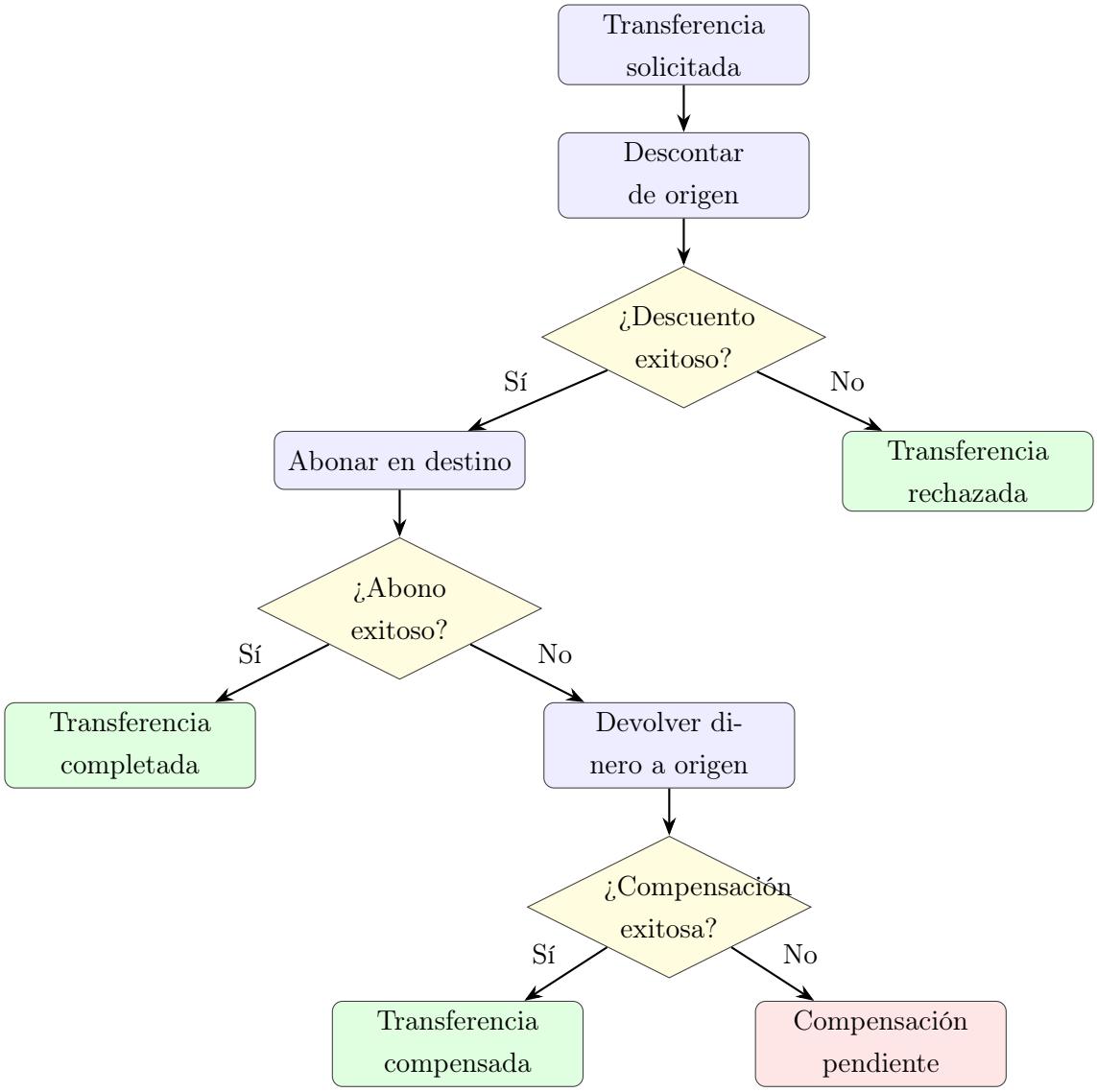


Figura 6: Estados de una transferencia coordinada mediante una saga

3. Evidencia de fallos durante la ejecución

3.1. Error de conexión

En la siguiente captura aparece el error ECONNRESET este mensaje indica que la conexión se cerró o se reinició antes de que la solicitud terminara normalmente

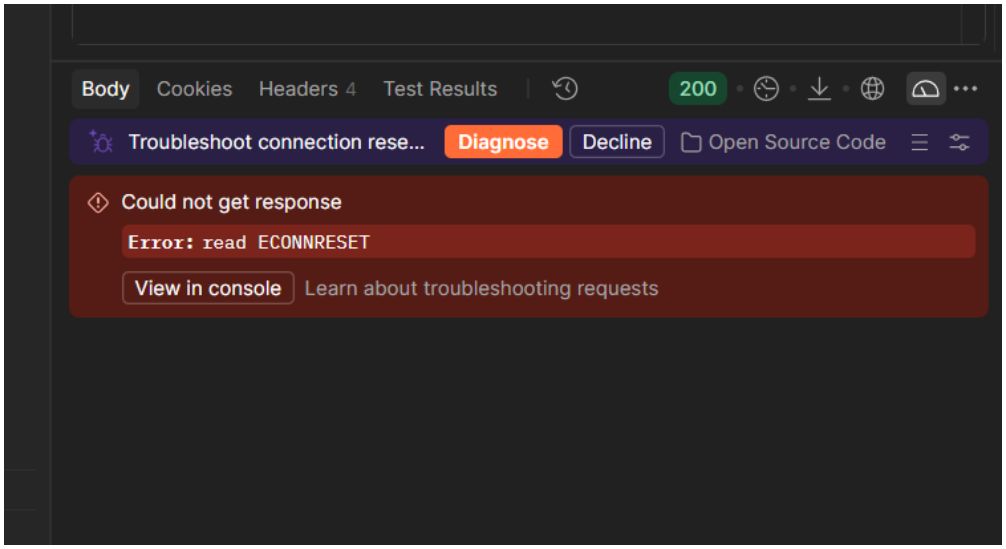


Figura 7: Error de conexión ECONNRESET.

Este tipo de error puede impedir que el coordinador reciba la respuesta del servicio llamado pero no permite asegurar por sí solo si la operación remota llegó a ejecutarse

3.2. Servicio no disponible

La siguiente captura muestra el error HTTP 503 Service Unavailable, esto indica que el servicio no pudo atender la solicitud en ese momento.

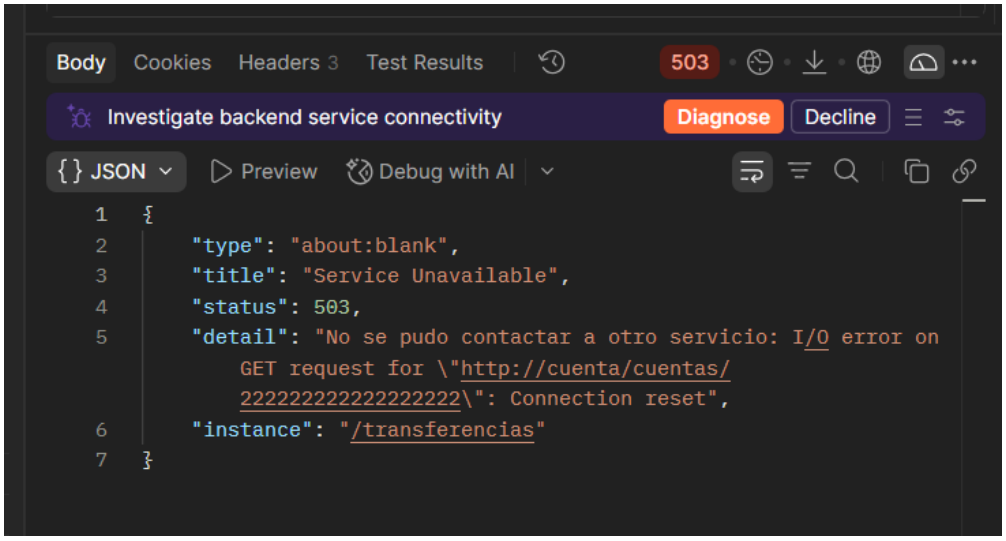


Figura 8: Error HTTP 503 durante la comunicación entre servicios

Si el servicio de destino no puede completar el abono después de que el descuento se realizó la saga debe intentar la compensación para evitar que el dinero permanezca descontado.

3.3. Saldos observados antes de la compensación

Las siguientes consultas muestran un saldo de $700 para Ana y $700 para Beto.

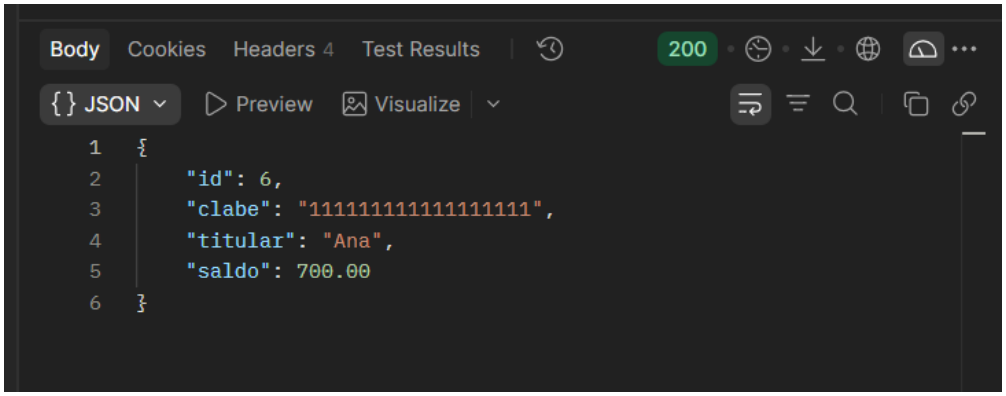


Figura 9: Saldo observado de Ana: $700

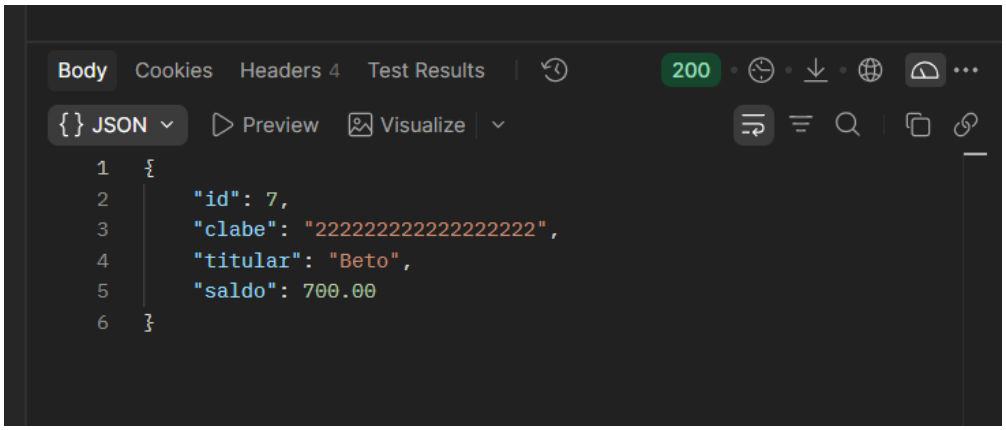


Figura 10: Saldo observado de Beto: $700

Estos valores corresponden a las consultas que aparecen en las capturas

3.4. Error interno durante la transferencia

En otro intento se recibió el error HTTP 500 Internal Server Error

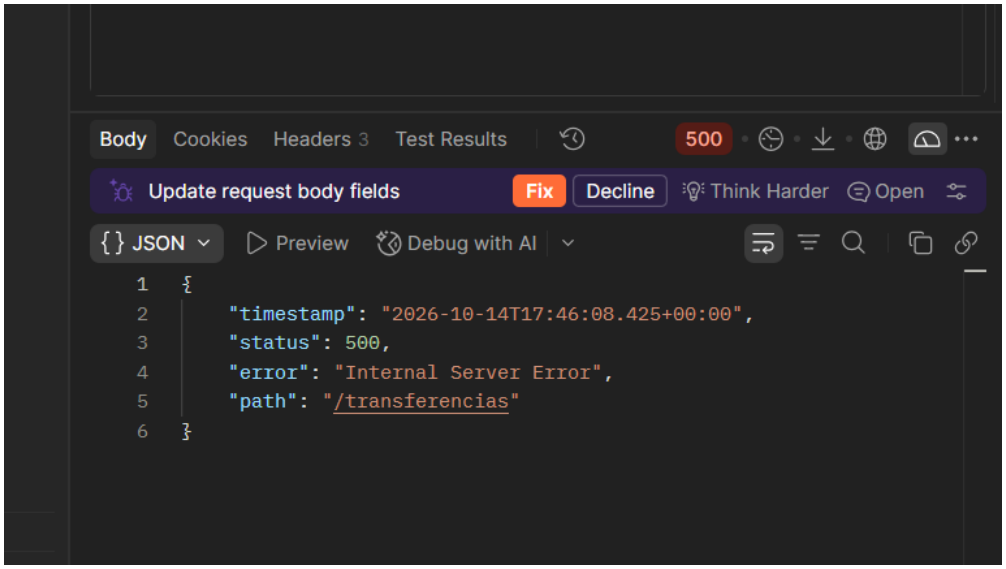


Figura 11: Error HTTP 500 durante una solicitud de transferencia.

Este error indica que ocurrió un problema interno en el servidor para poder determinar si es necesario compensar la operación se debe identificar si el descuento se había completado antes de que apareciera el error.

4. La compensación funcionando

4.1. Respuesta de la saga

La siguiente captura muestra una respuesta HTTP 422 Unprocessable Entity. El mensaje indica que el abono falló pero que el dinero fue devuelto a la cuenta de origen y también muestra que el fallo fue provocado por la variable CLABE_FALLA_ABONO.

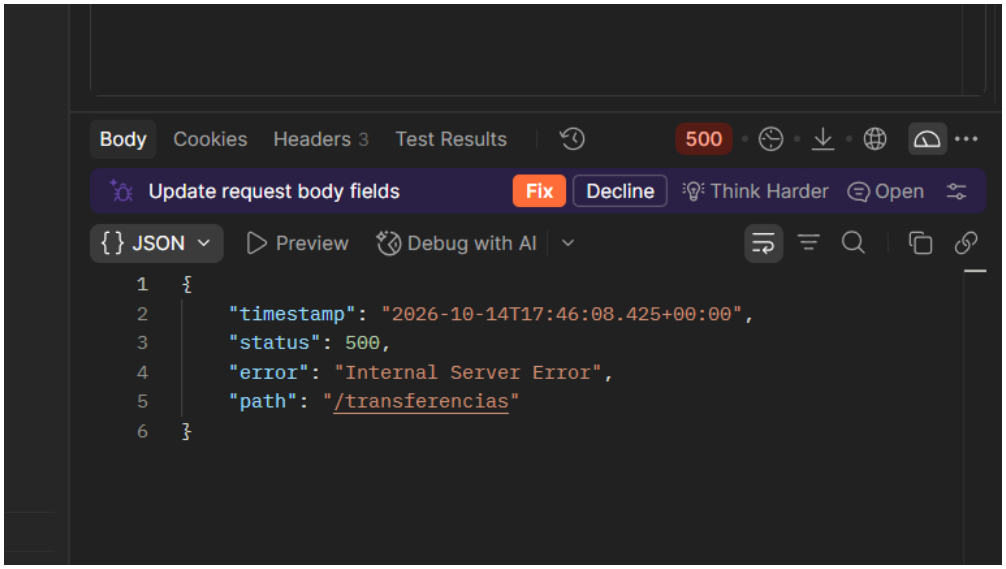


Figura 12: Respuesta HTTP 500 durante el intento de transferencia.

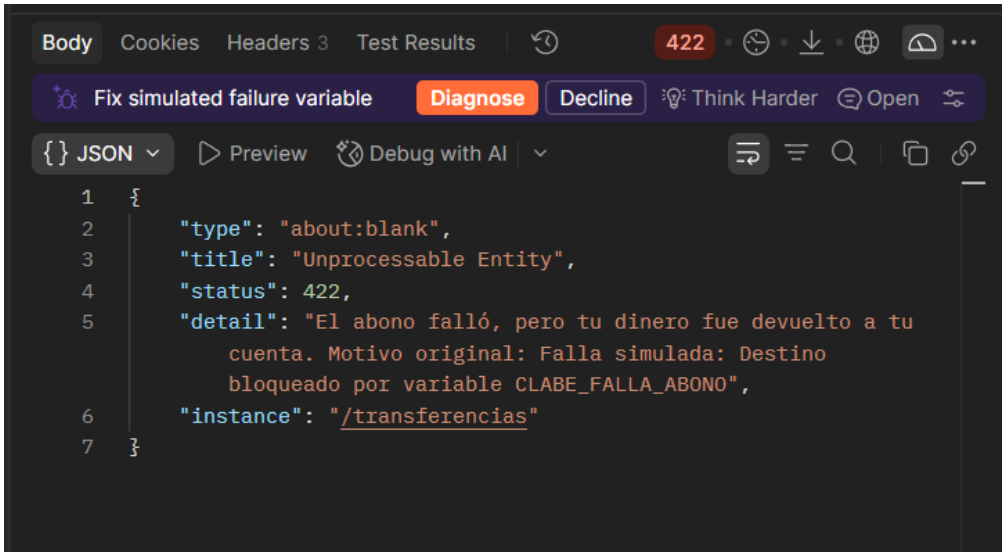


Figura 13: Respuesta HTTP 422 que informa del fallo del abono y de la devolución del dinero.

La respuesta permite observar que el sistema identificó el fallo del abono y comunicó que se había devuelto el dinero

1. Se intentó descontar el dinero de la cuenta de origen.
2. Se intentó realizar el abono en la cuenta de destino.
3. La falla simulada impidió completar el abono.
4. La saga intentó devolver el dinero a la cuenta de origen.
5. La API informó que el abono falló y que el dinero fue devuelto.

La respuesta HTTP por sí sola no permite comprobar los saldos finales para verificar numéricamente la compensación es necesario consultar ambas cuentas después de la operación.

4.2. Saldos después de compensar

Si los saldos anteriores a la operación fallida eran de $700 para Ana y $700 para Beto y la transferencia fallida era de $100 el resultado esperado después de una compensación correcta sería:

Cuenta	Antes de la operación fallida	Después de compensar
Ana	$700.00	$700.00
Beto	$700.00	$700.00

Tabla 2: Saldos esperados después de una compensación correcta.

Estos valores es el resultado esperado deben confirmarse mediante nuevas consultas a la base de datos pero no representan una medición posterior comprobada en las capturas disponibles.

5. Cuando la compensación también falla

La compensación también puede fallar esto puede ocurrir si el servicio de la cuenta de origen deja de responder o si aparece un error al intentar devolver el dinero.

Si el descuento de $100 ya se realizó y la compensación no se completa la cuenta de origen podría permanecer con un saldo de $600.00 mientras la cuenta de destino conserva $700 estos valores son un ejemplo del posible estado inconsistente y no un resultado observado en las capturas.

Para resolver esta situación se necesita:

1. Registrar la compensación como pendiente
2. Conservar un identificador único de la transferencia
3. Reintentar la devolución cuando el servicio vuelva a estar disponible
4. Verificar el estado de la operación antes de repetir el movimiento
5. Marcar la transferencia como compensada solo después de confirmar la devolución

El identificador único permite reconocer la misma transferencia cuando se realiza un reintento y también nos ayuda a evitar que una devolución se aplique dos veces esto en un sistema real se puede implementar en un proceso para que revise las compensaciones pendientes y genere alertas cuando no puedan resolverse automáticamente.

6. Código clave

6.1. Configuración del Circuit Breaker en application.yml

Para configurar el Circuit Breaker se utilizó Resilience4j en el archivo application.yml se definieron los parámetros de la instancia cuentaCB la ventana de evaluación considera cuatro llamadas el tiempo de espera en estado abierto es de 15 segundos y en estado semiabierto se permiten dos llamadas de prueba.

```
resilience4j:
  circuitbreaker:
    instances:
      cuentaCB:
        slidingWindowSize: 4
        waitDurationInOpenState: 15s
        permittedNumberOfCallsInHalfOpenState: 2
```

Listing 1: Configuracion de Resilience4j en application.yml

La ventana de cuatro llamadas permite evaluar los resultados recientes de las solicitudes, si se alcanza el umbral de fallos configurado el circuito puede abrirse y después de 15 segundos puede pasar al estado semiabierto y permitir dos llamadas de prueba para comprobar si el servicio se recuperó.

6.2. Anotaciones en el cliente HTTP en CuentaCliente.java

En el archivo CuentaCliente.java se agregó la anotación @CircuitBreaker(name = çuentaCB") a los métodos que realizan las operaciones con el servicio de cuentas.

```
@CircuitBreaker(name = "cuentaCB")
public Cuenta consultar(String clabe) {
}

@CircuitBreaker(name = "cuentaCB")
public void cargar(String clabe, BigDecimal monto) {
}

@CircuitBreaker(name = "cuentaCB")
public void abonar(String clabe, BigDecimal monto) {
}
```

Listing 2: Anotaciones integradas en CuentaCliente.java

Los tres métodos utilizan la misma instancia del Circuit Breaker de esta manera las llamadas a consulta, cargo y abono quedan sujetas a la configuración de cuentaCB

Cuando el circuito se encuentra abierto las llamadas protegidas pueden ser rechazadas sin intentar acceder a la dependencia esto ayuda a evitar solicitudes repetidas mientras el servicio presenta fallos.

6.3. Mapeo de puerto en docker-compose.yml

También se configuró el puerto del servicio de transferencia en el archivo docker-compose.yml ,se estableció el mapeo del puerto 8082 del equipo anfitrión al puerto 8080 del contenedor.

```
  transferencia:
    ports:
      - "8082:8080"
    <<: *servicio
    build: ./transferencia
```

Listing 3: Mapeo del puerto en docker-compose.yml

Con esta configuración el servicio puede recibir solicitudes desde el equipo anfitrión mediante el puerto 8082. Dentro del contenedor el servicio continúa utilizando el puerto 8080

7. Circuit Breaker: evidencia de los tres estados

7.1. Estado CLOSED

En el estado CLOSED las solicitudes pueden pasar normalmente el circuito registra los resultados de las llamadas para detectar si se presenta una cantidad importante de fallos.

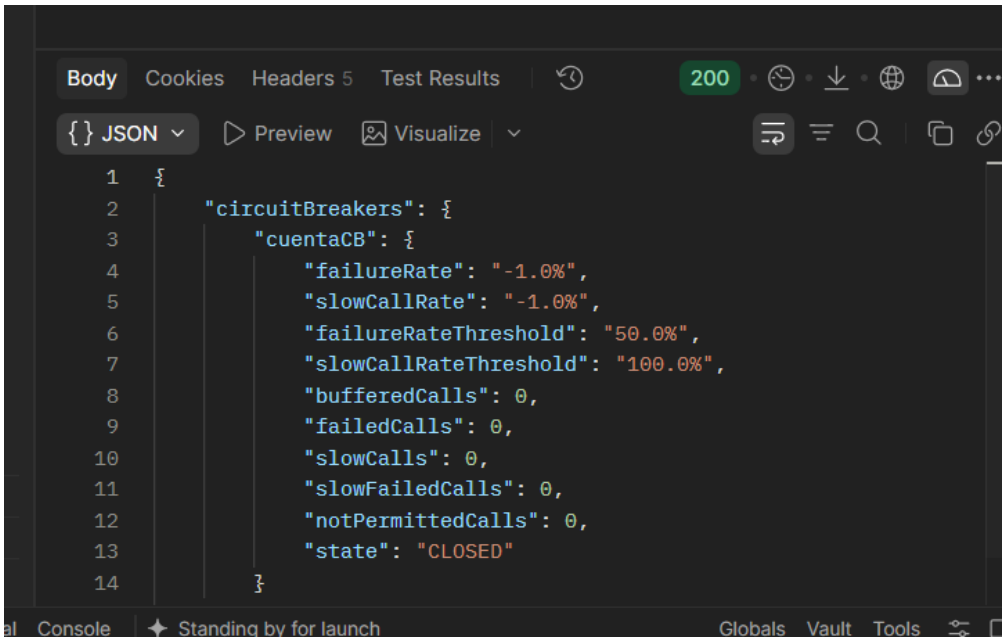


Figura 14: Circuit Breaker en estado CLOSED.

La captura muestra el circuito cerrado resaltando que en este estado el sistema permite las llamadas al servicio protegido

7.2. Estado OPEN

Cuando el circuito detecta suficientes fallos puede pasar al estado OPEN en este estado las llamadas protegidas se rechazan temporalmente sin intentar comunicarse con el servicio

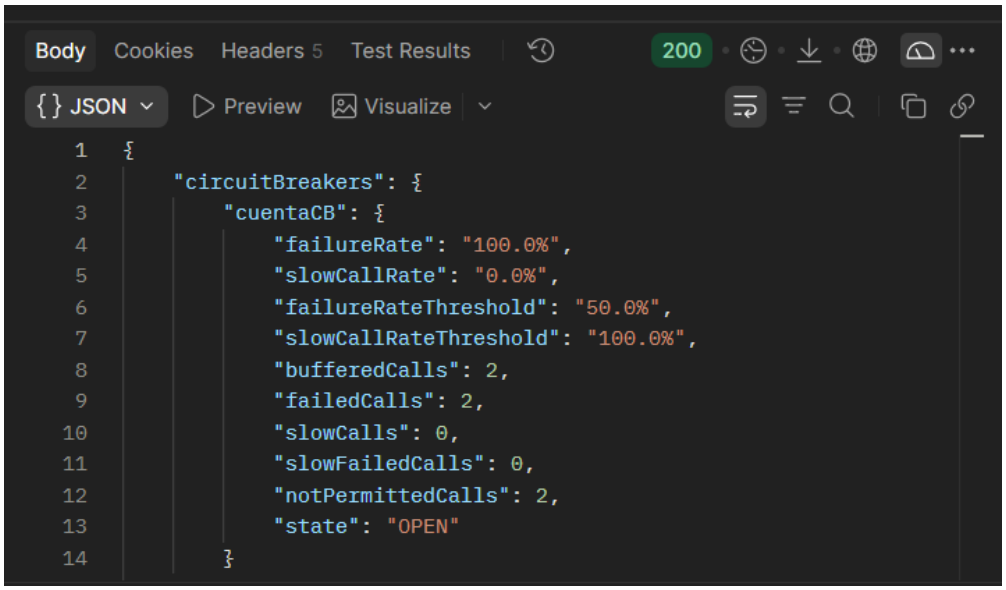


Figura 15: Circuit Breaker en estado OPEN.

De esta manera se evita seguir realizando solicitudes que podrían terminar en error

7.3. Estado HALF_OPEN

Después del tiempo de espera el circuito puede pasar al estado HALF_OPEN en este estado permite un número limitado de llamadas para comprobar si el servicio volvió a responder correctamente.

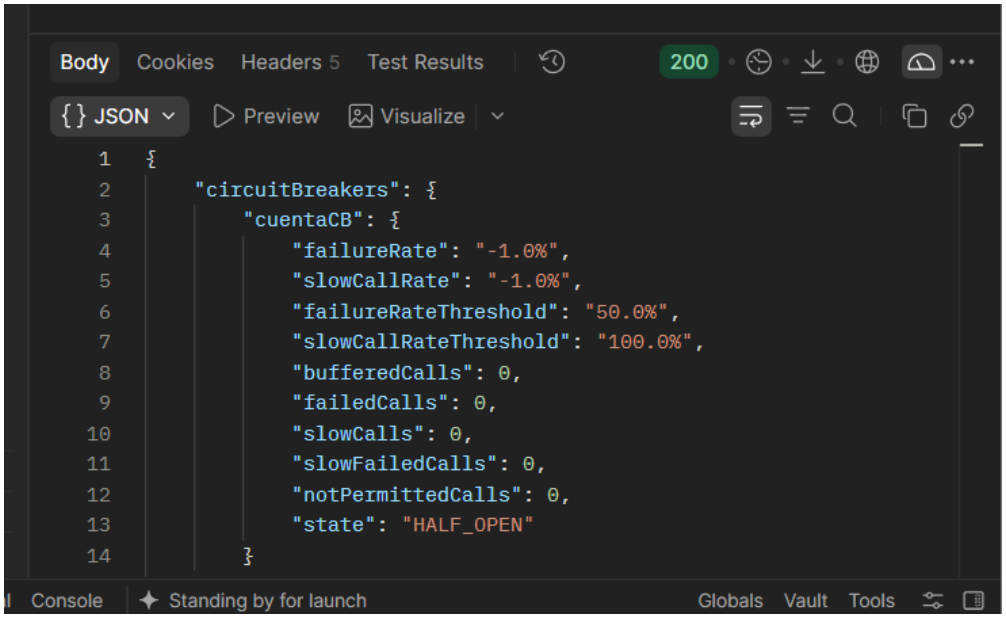


Figura 16: Evidencia del estado HALF_OPEN.

Si las llamadas de prueba tienen éxito el circuito puede volver a CLOSED Si vuelven a fallar puede regresar a OPEN.

8. Lo que la saga no garantiza: aislamiento

La saga permite recuperar el estado de una transferencia mediante operaciones de compensación pero esto no garantiza por sí sola el aislamiento de una transacción atómica de base de datos. Por ejemplo si se descuentan $100 de una cuenta que tenía $700 una persona que consulte el saldo antes de la compensación podría observar $600 si después falla el abono y la devolución se completa el saldo volvería a $700

Esto significa que otros clientes pueden observar un estado intermedio antes de que la saga termine, la compensación corrige el estado posterior pero no elimina las lecturas que ya se hicieron, para reducir este problema se pueden registrar estados como PENDING, COMPLETED y COMPENSATION_PENDING, también se pueden utilizar identificadores únicos para las transferencias y controlar cómo otros procesos consultan los movimientos que todavía no han terminado.